

CAS4133 LLM · 기말 대비 · Chapter #10

10. LLM의 도구 사용 (Tool Use with LLMs)

Prompting 기반 tool use(function calling)부터, 학습 기반 방법인 Toolformer(self-supervised, loss 기반 filtering)와 ToolLLM(16,464 real-world APIs, DFS 기반 데이터 수집)까지

 강의일: 2026-05-18(도입) / 2026-05-20(본강의) 예상 비중: 중상 (Agents의 기반 챕터) 분량: 본문 7섹션 + 예상문제

목차

1. 왜 Tool Use인가 — LLM의 한계와 동기
2. Prompting 기반 접근 (Function Calling & skill.md)
3. Toolformer — 핵심 아이디어와 표기법
4. Toolformer — 데이터 생성 5단계 (filtering 수식)
5. Toolformer — 결과와 한계
6. ToolLLM — RapidAPI · Instruction 생성 · DFSDT
7. ToolLLM — Retriever · ToolLLaMA · 평가 + 비교표
8. 예상문제 (T/F + Pick-all)

정의

Tool use: LLM이 외부 도구(external tools) — 계산기(calculator), 검색 엔진, 날씨 API 등 — 를 호출하여, 파라미터 내부 지식(intrinsic/parametric knowledge)만으로는 할 수 없는 일을 수행하는 능력. **Tool use is often considered as a core functionality of AI agents.**

LLM 단독으로는 잘 못 하는 것들이 있다:

- **정확한 계산** — 언어모델은 산술 연산을 토큰 패턴으로 흉내낼 뿐, 신뢰할 수 있는 계산기가 아님.
- **최신/실시간 정보** (real-time information) — 예: "What is the weather like in Paris today?"는 실시간 API 접근 없이는 정확히 답할 수 없고, 그냥 답하면 **hallucination**이 됨.
- **외부 세계에 대한 행동** — 이메일 전송, 파일 조작 등 텍스트 생성 밖의 action.

교수 강조 (강의 발언)

"솔직히 검색(retrieval)도 그런 도구 중 하나죠? 그래서 RAG 시스템은 이제 tool use를 하는 LLM에 포섭됩니다. ... '**LLM agent**'라고 부르려면 ① **tool use**, ② **memory**, ③ **planning & action**(+ 어느 정도의 reflection)을 갖춰야 합니다. Tool use는 특히 LLM의 실세계 배치(real-world deployment)에 매우 중요합니다."

★ 시험 핵심

- RAG의 retrieval은 tool use의 한 특수 사례로 볼 수 있다 ($\text{retrieval} \subset \text{tool use}$).
- Agent의 3대 구성요소: **tool use / memory / planning & action** — #11 LLM Agents 챕터로 이어지는 연결고리.
- Tool use를 가능하게 하는 두 갈래: **prompting-based**(in-context learning) vs **training-based**(Toolformer, ToolLLM).

⚠ 함정 포인트 (T/F 대비)

- "Tool use is unrelated to RAG; they are completely separate paradigms." → **False**.
The professor explicitly noted that retrieval is one kind of tool, so RAG is subsumed

under tool-using LLMs. Also, ToolLLM's inference pipeline is explicitly analogous to RAG (retrieve APIs → generate with them).

- "An LLM that can use tools is automatically an agent." → **False**. Tool use is only one of the core functionalities of agents (alongside memory, planning & action).

Prompting 기반 접근 (Prompting-based Approach / Function Calling)

가장 직관적인 방법: 도구에 대한 정보를 **in-context learning**으로 모델에게 알려준다. 도구의 명세 (specification)를 프롬프트에 넣어주면, LLM이 사용자 질의에 따라 **적응적으로(adaptively)** 어떤 도구를 언제 호출할지 결정한다. (OpenAI function calling 가이드의 `get_weather` 예시)

Function Calling 동작 5단계 (OpenAI 예시)

- 1 **Call model with functions defined** — `tools` 인자(argument)로 함수 목록을 정의하여 모델에 전달. 각 함수는 `name( get_weather )`, `description("Get current temperature for a given location")`, `parameters( location )`로 구성.
- 2 **Model decides to call function(s)** — 모델이 호출할 **함수 이름과 입력 인자를 반환**. (사용자 질의 "weather in Paris"를 파싱해 `location` 인자를 추출)
- 3 **Execute function code** — 모델 응답을 파싱하여 **개발자 측에서** 실제 함수 코드를 실행. (LLM이 직접 실행하는 것이 아님!)
- 4 **Supply model with results** — API 결과(예: 13°C)를 다시 모델 입력 컨텍스트로 삽입.
- 5 **Model responds** — 결과를 통합(incorporate)한 최종 응답 생성.

내부 동작: System Message + Detailed Description

`tools` 리스트를 넣으면 내부적으로는 **시스템 메시지(system message)**로 변환된다: "You have access to the following tools" + 각 도구의 **이름(tool name)**과 **자연어 설명(tool description)**이 시스템 프롬프트에 덧붙는다. LLM은 오직 이 자연어 단서만 보고 어떤 API를 언제·어떻게 호출할지 판단한다.

📖 교수 강조 — Claude skill.md 비유

"이것은 최근 **Claude 모델이 skill.md 파일을 사용하는 방식과 매우 유사**합니다. skill.md에는 스킬 이름, 설명, 그리고 경우에 따라 사용 예시가 포함되는데, 이는 도구 정의와 정확히 동일하고 **주된 차이는 API의 존재 여부뿐**입니다. ... 또, `multiply` 함수의 이름을 'addition'이라고 잘못 지으면 이름과 설명이 일관되지 않아 LLM이 혼란스러워합니다. 그래서 **일관된 이름(consistent naming)**과 **상세한 설명(detailed description)**이 매우 중요합니다."

한계 → 학습 기반 접근의 필요성

구분	Prompting-based (in-context)	Training-based (fine-tuning)
장점	Flexibility — 새로운/맞춤형 API를 즉시 추가 가능 (학습 불가능한 새 API에는 이 방법뿐)	자주 쓰이는 고정된 API 집합 (Excel, calculator 등)에 대해 더 나은 precision & recall
단점	전적으로 LLM의 in-context learning 능력에 의존 → only reliable for large LLM , 작은 오픈소스 모델에선 suboptimal	주석(annotation) 비용, 새 도구가 추가 될 때마다 재학습 필요
적합 시나 리오	커스텀/신규 도구, 강력한 모델	자주 호출되는 고정 도구(frequently called tools), 작은 모델

★ 시험 핵심

Prompting 기반 tool use = **implicit in-context learning**. 슬라이드 원문: "this approach might be suboptimal, as only relying on implicit in-context learning by LLM (**may be only reliable for large LLM**), when there are tools frequently called and hence should be handled well." → 자주 쓰이는 도구는 **training level**에서 배우는 게 낫다는 것이 Toolformer/ToolLLM의 출발점.

⚠ 함정 포인트 (T/F 대비)

- "In function calling, the LLM directly executes the external API code." → **False**. The model only returns the function **name and input arguments**; the developer-side code parses the response and executes the function, then feeds the result back.
- "The tool name has little effect as long as the description is detailed." → **False**. The LLM recognizes tools purely from natural-language cues, so an inconsistent name (e.g., naming a multiply function "addition") confuses the model. Both consistent naming and detailed description matter.

- "Prompting-based tool use is always inferior to training-based tool use." → **False**.
For new or customized APIs, prompting is the only feasible option (flexibility);
training is preferable mainly for a fixed set of frequently used tools.

Toolformer — 핵심 아이디어와 표기법 (Schick et al., NeurIPS 2023 Oral)

동기: SFT의 비용 문제

도구 사용을 학습시키는 가장 자연스러운 방법은 **supervised fine-tuning (SFT)** — Self-RAG(Asai et al., ICLR 2024 Oral)처럼 human annotator나 large LLM으로 (instruction → tool) 매핑 데이터를 주석하는 것. 그러나:

- 주석에 **large cost**가 필요하고, 도구는 계속 진화/추가되므로 일반 생성 과제보다 더 까다로움.
- large LLM을 annotator로 쓰면 그 LLM의 **intrinsic knowledge**에 **전적으로 의존**하게 됨.

정의 — Toolformer의 기여

Self-supervised learning for tool use: 사람 주석 없이(without human annotation) 데이터로부터 과제를 설계. 핵심 아이디어 = **"Useful tool → Helpful to predict next token"**. 도구 호출이 **next-token prediction loss**를 낮춰주면 유용한 호출, 높이면 무효한 호출.

교수 강조 — Steel City 예시

"Pittsburgh is also known as *the Steel City*"라는 코퍼스 문장에서 "the Steel City"를 예측한다고 하자.

- ① API 질의 "What other name is Pittsburgh known by?" → 결과 "Steel City"를 컨텍스트에 덧붙이면 → **이미 답을 조건으로 줬으니 loss가 크게 감소** (성공적 도구 호출).
- ② 질의 "Which country is Pittsburgh in?" → 결과 "United States"를 덧붙이면 → 무관한/방해되는(distracting) 정보라 **loss가 오히려 증가할 위험** (무효한 도구 호출).
- ③ 호출했지만 결과가 안 돌아오면(failed call) 빈 반환값 — query만 컨텍스트에 남음.

표기법 (Notation)

API 호출의 표현

$$c = (a_c, i_c)$$

a_c : API의 이름 (name of API), i_c : 해당 입력 (corresponding input). API 호출은 특수 토큰 $\langle \text{API} \rangle$, $\langle / \text{API} \rangle$, $\rightarrow$ 를 사용해 일반 텍스트 토큰으로 직렬화(linearize)된다:

- 결과 없는 표현: $e(c) = \langle \text{API} \rangle a_c(i_c) \langle / \text{API} \rangle$
- 결과 포함 표현: $e(c, r) = \langle \text{API} \rangle a_c(i_c) \rightarrow r \langle / \text{API} \rangle$ — 화살표 $\rightarrow$ 오른쪽이 **result of API call** r .

★ 시험 핵심

- Toolformer = **self-supervised** (no human annotation). Self-RAG처럼 LLM을 annotator로 활용하되, **채택 여부는 loss 감소라는 자동 기준**으로 판단한다는 점이 차별점.
- API 호출이 텍스트 토큰 시퀀스로 표현되므로 **일반 LM 목적함수(next-token prediction)로 그대로 학습 가능**.
- 특수 토큰 3종: $\langle \text{API} \rangle$ / $\langle / \text{API} \rangle$ / $\rightarrow$.

⚠ 함정 포인트 (T/F 대비)

- "Toolformer requires human-annotated examples of correct tool calls for fine-tuning." $\rightarrow$ **False**. It is self-supervised: candidate API calls are sampled by the LM itself via in-context learning and kept only if they reduce next-token prediction loss.
- "In Toolformer, a tool call is judged useful if the API returns a factually correct answer." $\rightarrow$ **False**. Usefulness is judged purely by whether conditioning on the call (and its result) **reduces the next-token prediction loss** — no factuality check is performed.

Toolformer — 데이터 생성 파이 (Converting the Pre-training Corpus) 프라인 5단계

목표: 주어진 **사전학습 코퍼스**(예: Wikipedia/web corpus)를 API 호출로 **증강(augment)**하여 학습 데이터로 변환.

- 1 Sampling API Calls** (through in-context learning) — LLM에게 few-shot 프롬프트(예: "Joe Biden was born in Scranton" → [QA(Where was Joe Biden born?) → Scranton] 식의 예시)를 주고, 코퍼스 각 위치에 후보 API 호출 c_i 를 샘플링하게 함. LLM을 *annotator*로 활용 (Self-RAG와 유사). 이 단계 산출물은 노이즈가 많음.
- 2 Executing API Calls** — 각 후보 호출 c 를 실제로 실행하여 출력 r 을 얻음.
- 3 Filtering API Calls** — loss 비교 기준으로 유용한 호출만 남김 (아래 수식).
- 4 Model fine-tuning** — 필터링을 통과한 API 호출이 삽입된 증강 코퍼스로, **표준 next-token prediction**(언어모델링) 목적함수로 미세조정.
- 5 Inference** — 생성 중 모델이 스스로 `<API>` 와 질의, 그리고 `→` 를 생성하면 **디코딩을 중단**하고, 질의를 외부 API에 전달 `→` 결과를 받아 `→` 오른쪽에 삽입 `→` 생성 재개. (Search-R1이 special token에 retrieval 결과를 삽입하는 방식과 매우 유사)

Step 3 Filtering 수식 — 가장 중요한 출제 포인트

가중 next-token prediction loss (position i = API 호출 위치)

$$L_i(\mathbf{z}) = - \sum_{j=i}^n w_{j-i} \log p_M(x_j \mid \mathbf{z}, x_{1:j-1})$$

$\mathbf{z}$: 위치 i 앞에 prefix로 삽입되는 시퀀스, w_{j-i} : 호출 위치에서 멀어질수록 감소하는 가중치 (교수: "가중치는 추가 성능 향상을 위한 하이퍼파라미터일 뿐, 무시해도 된다"), p_M : 모델 M 의 토큰 확률.

두 가지 손실의 정의 (ε : empty sequence)

$$L_i^+ = L_i(e(c_i, r_i))$$

$$L_i^- = \min \left(L_i(\varepsilon), L_i(e(c_i, \varepsilon)) \right)$$

L_i^+ : **with function call and response** — 호출+결과를 모두 조건으로 한 loss.

L_i^- : 두 경우의 **최솟값(min)** — ① $L_i(\varepsilon)$: **no function call** (아무 호출도 안 함), ② $L_i(e(c_i, \varepsilon))$: **no response** (호출은 했지만 결과가 비어 있음, 즉 query만 컨텍스트에 추가).

Filtering 기준 (threshold τ)

$$L_i^- - L_i^+ \geq \tau \implies \text{keep the API call}$$

"API 호출+결과 추가가, **호출을 전혀 안 하거나 결과를 못 받은 경우에 비해** loss를 **적어도 τ 만큼** 줄일 때만" 데이터로 유지. $\tau = 0$ 이면 조금이라도 도움 되는 호출은 전부 채택, τ 가 클수록 확실히 도움 되는 호출만 채택(더 적고 더 깨끗한 데이터). τ 는 개발자 측 하이퍼파라미터.

📖 교수 강조 — 왜 $L_i(e(c_i, \varepsilon))$ (query-only) 항이 필요한가?

"LLM에게는 **질의 텍스트 자체도 예측에 영향**을 줍니다. 질의가 관련 있으면 결과 없이 질의만 있어도 next-token 예측에 도움이 될 수 있죠. 그래서 '결과 덕분에 좋아졌다'는 **더 명확한 보장**을 위해, 결과 없는 query-only 경우보다도 τ 이상 좋아야만 채택하도록 min에 포함시킨 것입니다."

★ 시험 핵심

- 비교 방향 주의: $L_i^- - L_i^+ \geq \tau$ (loss는 작을수록 좋으므로 L_i^+ 가 충분히 작아야 채택).
 $L_i^+ - L_i^- \geq \tau$ 로 뒤집어 내면 함정!
- L_i^- 는 단순히 "no call" loss가 아니라 **min(no call, call-but-no-response)**이다.
- fine-tuning은 특별한 목적함수가 아니라 **증강 코퍼스에 대한 표준 LM(next-token prediction) loss**.
- Inference에서 모델이 $\rightarrow$ 를 생성하는 순간 디코딩 중단 $\rightarrow$ API 실행 $\rightarrow$ 결과 삽입 $\rightarrow$ 재개. 결과 텍스트는 **모델이 생성하는 것이 아니라 API에서 온다**.

⚠ 함정 포인트 (T/F 대비)

- "Toolformer keeps an API call when $L_i^+ - L_i^- \geq \tau$." $\rightarrow$ **False**. The criterion is $L_i^- - L_i^+ \geq \tau$: the call is kept when it **reduces** the loss by at least τ compared to no call / no response.

- " L_i^- is the loss when no API call is made at all." → **False (incomplete)**. $L_i^- = \min(L_i(\varepsilon), L_i(e(c_i, \varepsilon)))$ — it also covers the case where the call is made but returns no result (query-only context).
- "Setting a larger τ keeps more API calls in the augmented dataset." → **False**. A larger τ is a stricter margin, so **fewer** calls survive filtering.
- "During inference, Toolformer generates the API result tokens itself." → **False**. Decoding pauses at " $\rightarrow$ "; the result is obtained from the external API and inserted, then generation resumes.
- "Filtering compares the loss only on the single next token." → **False**. The loss is a **weighted sum over the subsequent tokens** $j = i, \dots, n$ (with decaying weights w_{j-i}).

실험에 사용된 5가지 도구

도구	역할
Question Answering (QA)	사실 질의에 대한 답 반환 (예: "Where was Joe Biden born?" → Scranton)
Wikipedia Search	위키피디아 검색 결과 반환
Calculator	산술 계산
Calendar	현재 날짜 반환
Machine Translation (MT)	다른 언어 텍스트를 영어로 번역

교수: "다섯 가지 도구에 한정되는 방법론은 아니지만, 실험에서는 이 다섯 가지만 사용됩니다." (이유는 아래 '한계' 참조)

결과 (GPT-J 6B)

- 베이스 모델: **GPT-J — 당시 6B(60억) 규모의 오픈소스 LLM**.
- Tool use 덕분에 훨씬 작은 모델임에도 여러 QA 과제에서 **GPT-3를 능가** — "Tool use makes significant improvement for various tasks".
- **공정한 비교(ablation)**: 증강 코퍼스로 fine-tuning만 한 효과를 분리하기 위해, **tool use를 비활성화(disabled)한 Toolformer**도 비교. → fine-tuning 자체는 성능을 약간만 올리고, 주된 향상은 도구 호출 자체에서 나옴.

한계 (Limitations) → ToolLLM의 동기

- 핵심 단계(코퍼스에 API 호출 증강)가 **전적으로 LLM의 in-context learning에 의존** → 그 자체의 문제(노이즈·신뢰성)를 그대로 가짐.

- 데이터 증강 비용이 큼: 후보 샘플링 + 실제 API 실행 + loss 2회 계산을 코퍼스 전 위치에 대해 수행.
- **증강 비용이 도구 개수에 비례해 증가** → 도구를 5개만 쓴 이유. 그러나 현실 세계에는 도구가 엄청나게 많다 (ToolLLM: ~3,000 tools / ~16,000 APIs 규모). → **real-world scale로 확장 불가**.
- (구조적 한계) 각 호출을 독립적으로 필터링하므로 여러 도구를 연쇄(chain)하는 multi-tool 시나리오를 다루지 못함.

★ 시험 핵심

"smaller GPT-J (6B) + tools > much larger GPT-3 (without tools)" 구도 + "main gain comes from tool calls, not from fine-tuning itself" (disabled-tool ablation) — 두 문장 모두 T/F 단골 소재.

⚠ 함정 포인트 (T/F 대비)

- "Toolformer's improvement mainly comes from fine-tuning on additional data, and tool calls at inference give only a marginal gain." → **False**. The ablation with tool use disabled shows fine-tuning helps only slightly; the main improvement comes from the tool calls themselves.
- "Toolformer scales easily to thousands of tools." → **False**. Augmentation cost grows with the number of tools, which is why only 5 tools were used — this is precisely the motivation for ToolLLM.
- "Toolformer was trained on GPT-3." → **False**. It fine-tunes GPT-J (6B, open-source); GPT-3 is the larger baseline it outperforms on several QA tasks.

정의 — ToolLLM의 목표

Scalable instruction tuning for tool use capability of open-sourced LLMs: 오픈소스 LLM에 실세계 규모의 tool use 능력을 부여하기 위한 **데이터 수집 파이프라인**을 설계하고, **강력한 closed LLM(ChatGPT)을 annotator로 활용**해 주석 절차를 자동화한다. (방향은 SFT/instruction tuning으로 회귀하되, 수집을 자동화)

① API Collection — RapidAPI

- **RapidAPI:** 개발자와 수천 개의 실세계 API를 연결하는 **API marketplace**.
- 여기서 필터링을 거쳐 **3,451 tools / 16,464 APIs** 수집 — 즉 **하나의 tool이 여러 API를 가질 수 있는 계층 구조(hierarchy)** (tool $\supset$ API).

② Instruction Generation — ChatGPT로 지시 생성

도구 사용을 **요구하는** instruction을 ChatGPT로 생성. 두 측면에 집중: **(1) diversity**(다양한 실세계 도구), **(2) multi-tool usage**(한 지시가 여러 API를 호출).

- 1 **샘플링** — 전체 API 집합 S_{API} (16,464개)는 한 번에 프롬프트에 다 못 넣으므로, 매번 N 개의 API를 무작위 샘플링: $S_N^{sub} = \{API_1, \dots, API_N\}$.
- 2 **프롬프트 구성** — (1) task description + (2) 각 API의 **documentation** + (3) **3개의 few-shot 예시**. few-shot은 **single-tool용 12개 / multi-tool용 36개**의 다양한 예시 풀에서 매번 무작위 샘플링.
- 3 **생성** — ChatGPT가 S_N^{sub} 의 부분집합 $S_*^{sub} \subset S_N^{sub}$ 를 사용하는 instruction $Inst_*$ 들을 생성. (예: 부분집합이 calculator뿐이면 계산 질의, calculator+calendar면 둘 다 요구하는 질의)
- 4 **필터링 후** — 약 **200k qualified (instruction, relevant API) pairs** 확보 → 이 쌍이 **API retriever 학습**에 활용됨.

③ Solution Path Annotation — 유효성 검증 + DFSDT

생성된 instruction이 실제로 할당된 API로 풀리는지(매핑의 validity)를 확인하기 위해, ChatGPT에게 실제로 풀게 한다:

- 매 라운드 t 에서 ChatGPT가 이전 상호작용(action들과 API response r)에 기반해 action a_t 생성 → 행동 시퀀스 $\{a_1, \dots, a_N\}$.
- ChatGPT의 **function call 기능**을 사용: 샘플링된 S_N^{sub} 를 **system message 수준에서 available functions**로 제공.
- 시퀀스를 끝내기 위한 **추가 함수 2개: (1) "Finish with final answer" & (2) "Finish by giving up"**.
- 최종 답에 도달하면 → (instruction, API set, action sequence)를 **golden data**로 채택. 포기(give up)로 끝나면 → instruction과 API가 mismatch이거나 invalid한 질의 → **데이터에서 제거**.
- 탐색 효율을 위해 **DFS (depth-first search-based decision tree)** 사용: 한 행동 경로를 깊이 우선으로 끝까지 시도하고, 막히면 되돌아가(backtrack) 다른 분기를 시도. 단일 경로만 쫓는 CoT/ReAct식 순차 생성과 달리 **여러 분기를 탐색해 유효한 solution path를 더 잘 찾음**.

👤 교수 강조 (강의 발언)

"외부 API 서버가 실패할 수도 있으므로 시도가 거듭될 수 있습니다. 답을 만들면 golden data로 수집하고, 실패하면 통째로 버립니다. 이 시뮬레이션을 촉진하기 위해 효율적인 탐색용 **depth-first search**가 고려됩니다. 여기 나오는 action이나 **ReAct** 같은 개념은 **에이전트 강의(#11)에서 큰 부분**이 될 겁니다."

★ 시험 핵심

- 숫자 암기: **3,451 tools · 16,464 APIs (RapidAPI) · ~200k instruction-API pairs · few-shot 12(single)/36(multi) 중 3개 사용**.
- 품질 관리 방식 대비: Toolformer는 **loss 감소**로 필터링, ToolLLM은 **ChatGPT가 실제로 풀어 답에 도달하는지(solution path)**로 필터링.
- 종료 함수 2개(Finish with final answer / Finish by giving up)와 DFS의 역할(분기 탐색 · backtracking).

⚠ 함정 포인트 (T/F 대비)

- "ToolLLM filters its training data using the next-token prediction loss reduction criterion, like Toolformer." → **False**. ToolLLM validates data by having ChatGPT

actually solve the instruction; only paths that reach "Finish with final answer" are kept as golden data.

- "ToolLLM shows all 16,464 APIs to ChatGPT at once when generating instructions." → **False**. That is infeasible (each API needs name/description/documentation in context); ToolLLM subsamples N APIs at a time.
- "In ToolLLM, one tool corresponds to exactly one API." → **False**. RapidAPI has a hierarchy: one tool can contain multiple APIs (3,451 tools vs 16,464 APIs).
- "DFSDT generates a single chain of actions and stops when the model fails." → **False**. DFSDT explores multiple branches depth-first and can backtrack, unlike single-path CoT/ReAct-style generation; failure is handled by the explicit "give up" function and that data is discarded.
- "ToolLLM is a self-supervised method requiring no stronger LLM." → **False**. It relies heavily on a strong closed LLM (ChatGPT) for both instruction generation and solution path annotation.

추론 시 전체 구조 — RAG와의 유사성

테스트 instruction이 주어지면:

- 1 **Retrieving relevant tools & APIs** — API retriever로 top-K 관련 API 검색 (= RAG의 document retrieval에 대응).
- 2 **Generation with given APIs** — 검색된 API들을 컨텍스트로 받아 reasoning & API call의 action들을 생성 (= RAG의 generation에 대응).

API Retriever (Dense Retriever)

- BERT 기반 **dense retriever**를 학습 (DPR과 유사). **Query = instruction, Passage = API document**.
- **Positive pair**: instruction과 그에 할당된 relevant API. **Negative**: 다른 instruction이 사용하는 API.
- 결과: 토큰 기반 **BM25**는 물론, OpenAI의 임베딩 모델 **Ada**까지 능가 (It even outperforms the embedding model (Ada) provided by OpenAI).

📢 교수 강조 (강의 발언)

"흥미롭죠? BM25는 도구 호출에 특화돼 있지 않고 Ada도 일반(general) 임베딩 모델입니다. 그런데 **도구 호출은 매우 특수한 성질**을 가져서, 이 도메인에 대한 **fine-tuning**이 매우 도움이 됩니다."

ToolLLaMA & ToolEval

- **ToolLLaMA**: 수집한 instruction-solution path 쌍으로 **LLaMA(-2) 7B**를 **fine-tuning**한 모델. 일부 시나리오에서 당시 **ChatGPT**에 **필적하는(comparable)** 성능.
- 평가(ToolEval) 두 지표:
 - **Pass (rate)**: 주어진 지시를 LLM이 답할 수 있었는가 (answerability by LLM) — 제한된 횟수 안에 instruction을 성공적으로 완수한 비율.

- **Win (rate): ChatGPT(의 solution)와 비교한 품질** — 평가자가 둘 중 어느 답이 나은지 비교.
- **핵심 발견:** 학습된 retriever로 검색한 **top-5 API가, 데이터 생성에 쓰였던 ground-truth API 집합 S_N^{sub} 보다 오히려 더 좋은 성능** — 원래 집합은 N개 중에서 고른 것에 불과해, retriever가 **더 다양한 (diverse) 관련 API**를 찾아주기 때문.



필수 비교표 — Toolformer vs ToolLLM

구분	Toolformer (Schick et al., NeurIPS 2023 Oral)	ToolLLM (Qin et al., ICLR 2024)
학습 패러다임	Self-supervised learning (no human annotation)	Instruction tuning (SFT), ChatGPT를 annotator로 활용한 자동 파이프라인
데이터 생성	사전학습 코퍼스에 LM 자신의 in-context learning으로 API call 후보를 삽입(augment)	ChatGPT가 instruction 생성 + solution path annotation (function calling 사용)
품질 필터링 기준	Loss 기반: $L_i^- - L_i^+ \geq \tau$ (next-token loss를 τ 이상 줄이는 호출만 유지)	실행 기반: "Finish with final answer"에 도달한 solution path만 golden data로 유지 (give up → 폐기)
도구 수	5개 (QA, Wikipedia search, calculator, calendar, MT) — 증강 비용이 도구 수에 비례	3,451 tools / 16,464 APIs (RapidAPI, real-world scale)
탐색 방식	탐색 없음 — 각 위치의 단일 API call 후보를 독립적으로 평가	DFS DT (depth-first search decision tree, backtracking 가능)
Multi-tool	단일 호출 단위 (multi-tool 연쇄 미지원)	Multi-tool instruction을 명시적으로 생성 (few-shot 36개 풀)
Retrieval 단계	없음 (5개 도구 전부 학습에 포함)	BERT 기반 API retriever (DPR식; BM25-Ada 능가), 추론은 RAG와 유사한 2단계
베이스 모델	GPT-J (6B) → 여러 QA 과제에서 GPT-3 능가	LLaMA(-2) 7B → ToolLLaMA, ChatGPT에 필적

평가

downstream task 성능 (+ tool-disabled ablation)

ToolEval: **Pass rate / Win rate** (vs ChatGPT)

★ 시험 핵심

- ToolLLM 추론 = RAG 구조의 유추: **API retrieval ↔ document retrieval, action generation ↔ answer generation**.
- Retriever 학습 셋업(query=instruction, passage=API document, negative=다른 instruction의 API)과 **BM25·Ada보다 우수**.
- "**Retrieved top-5 APIs > ground-truth S_N^{sub}** " — 이유는 **diversity**. 직관에 반하는 결과라 출제 1순위.
- Pass = answerability(완수율), Win = ChatGPT 대비 품질 비교 — 두 지표의 정의 구분.

⚠ 함정 포인트 (T/F 대비)

- "Using the ground-truth API set used for data generation always upper-bounds the performance of retrieved APIs." → **False**. Retrieved top-5 APIs performed even **better** than the ground-truth S_N^{sub} , because retrieval allows more diversity (the original set was just a choice among N sampled APIs).
- "ToolLLM uses OpenAI's Ada embeddings as its API retriever." → **False**. It trains its own BERT-based dense retriever (DPR-style), which **outperforms** Ada (and BM25). Ada is a baseline, not the retriever.
- "Win rate measures whether the model can finish the instruction within a limited budget." → **False**. That is the **Pass** metric (answerability); **Win** compares solution quality against ChatGPT.
- "ToolLLaMA is a fine-tuned version of ChatGPT." → **False**. It is LLaMA(-2) 7B fine-tuned on the collected instruction-solution pairs; ChatGPT was only the data annotator / comparison target.

Q 예상문제 (직접 기출 없음 — 중간고사 이후 범위)

시험 형식: T/F는 정답 3점 / 오답 0점 / 무응답 1점. Pick-all은 "any wrong answer will lead to 0 points".

True / False

Q1. (True/False) 예상문제

In prompting-based tool use (function calling), the tool definitions inserted via the `tools` argument are internally converted into a natural-language description appended to the system message, and the LLM decides when to call an API via implicit in-context learning.

▼ 정답 & 해설 보기

True. The tools list becomes a system-level instruction ("You have access to the following tools") with each tool's name and description in natural language; the model relies on implicit in-context learning to decide whether/when to call, which is why this approach may be reliable only for large LLMs.

Q2. (True/False) 예상문제

Toolformer keeps an API call at position i if and only if $L_i^+ - L_i^- \geq \tau$, i.e., when conditioning on the call increases the next-token prediction loss by at least τ .

▼ 정답 & 해설 보기

False. The direction is reversed. The criterion is $L_i^- - L_i^+ \geq \tau$: the call (with its result) must **reduce** the loss by at least τ compared to making no API call or receiving no result. Since lower loss is better, L_i^+ must be sufficiently small.

Q3. (True/False) 예상문제

In Toolformer's filtering step, L_i^- is defined as the minimum of two losses: the loss with no API call at all ($L_i(\varepsilon)$) and the loss when the API call is made but returns an empty result ($L_i(e(c_i, \varepsilon))$).

▼ 정답 & 해설 보기

True. The query-only case is included because the call text itself can already help next-token prediction; taking the min gives a stronger guarantee that the improvement comes from the call *and its result*, not from the query text alone.

Q4. (True/False) 예상문제

Toolformer requires a separate reward model and human preference annotations to decide which API calls to include in its training corpus.

▼ 정답 & 해설 보기

False. Toolformer is **self-supervised**: candidate calls are sampled by the LM via in-context learning, executed, and filtered automatically by the loss-reduction criterion $L_i^- - L_i^+ \geq \tau$. No reward model and no human annotation are involved.

Q5. (True/False) 예상문제

At inference time, a Toolformer-trained model pauses decoding when it generates the " $\rightarrow$ " token, sends the generated query to the external API, inserts the returned result, and then resumes generation.

▼ 정답 & 해설 보기

True. The model itself generates `<API>`, the API name, the query, and " $\rightarrow$ " via next-token prediction (it was fine-tuned to do so), but the text on the right side of the arrow comes from the external API, not from the model — very similar to how Search-R1 inserts retrieval results at special tokens.

Q6. (True/False) 예상문제

ToolLLM validates each generated instruction by checking whether the resulting tool-call sequence reduces the next-token prediction loss on a pre-training corpus.

▼ 정답 & 해설 보기

False. That is Toolformer's criterion. ToolLLM validates data via **solution path annotation**: ChatGPT actually tries to solve the instruction with the assigned APIs (using its function-call feature and DFS-based search), and only sequences ending with "Finish with final answer" are kept as golden data; "Finish by giving up" cases are discarded.

Q7. (True/False) 예상문제

In ToolLLM's experiments, providing the model with the ground-truth API set S_N^{sub} (used to generate the test instruction) yields better performance than providing the top-5 APIs returned by the trained API retriever.

▼ 정답 & 해설 보기

False. Counter-intuitively, the **retrieved top-5 APIs perform even better** than the ground-truth set, because retrieval allows more diversity — the original set was merely a subset chosen from N randomly sampled APIs.

Q8. (True/False) 예상문제

Setting the Toolformer threshold τ to a larger value results in a smaller but higher-quality augmented dataset, since only API calls that reduce the loss by a larger margin are kept.

▼ 정답 & 해설 보기

True. With $\tau = 0$, any call that helps even slightly is added; a larger τ is a stricter margin, so fewer calls survive but each is more clearly useful. τ is a developer-side hyperparameter.

Pick all that are correct (any wrong answer → 0 points)

Q9. (Pick all that are correct) 예상문제

Which of the following are correct about **prompting-based tool use**?

- a. The LLM understands an API tool based on its tool name and natural-language description provided in the system prompt.
- b. After the model decides to call a function, it returns the function name and input arguments, and the developer-side code executes the function.
- c. It is preferable to training-based approaches when the target is a new or customized API, since training on such APIs is difficult.
- d. Because tool information is given in context, this approach works equally reliably for small open-source LLMs and large proprietary LLMs.

▼ 정답 & 해설 보기

정답: (a), (b), (c)

- (a) Correct. The tools list is converted into a system message ("You have access to the following tools") with names and descriptions; these natural-language cues are all the LLM sees.
- (b) Correct. The model only outputs the call (name + arguments); execution happens outside the model, and the result is supplied back.
- (c) Correct. Flexibility for new/custom APIs is the main advantage of the prompting-based approach.
- (d) Incorrect. It relies on implicit in-context learning, which may be reliable only for large LLMs; small models with limited in-context learning ability make this approach suboptimal.

Q10. (Pick all that are correct) 예상문제

Which of the following are correct about **Toolformer** (Schick et al., NeurIPS 2023)?

- a. Candidate API calls are sampled by prompting a language model with few-shot examples (in-context learning), then actually executed to obtain results.

- b. An API call is kept only when $L_i^- - L_i^+ \geq \tau$, where L_i^+ conditions on both the call and its response.
- c. After filtering, the model is fine-tuned with a specially designed tool-use objective that differs from standard language modeling.
- d. The fine-tuned GPT-J (6B) outperforms the much larger GPT-3 on several QA tasks thanks to tool use.

▼ 정답 & 해설 보기

정답: (a), (b), (d)

- (a) Correct. Step 1 (sampling via in-context learning) and Step 2 (executing API calls) of the pipeline.
- (b) Correct. This is exactly the filtering criterion; L_i^- is the min over the no-call and call-without-response cases.
- (c) Incorrect. Fine-tuning uses the **standard next-token prediction (language modeling) objective** on the augmented corpus; API calls are just text tokens with special markers.
- (d) Correct. Despite being far smaller than GPT-3, GPT-J with tool use surpasses it on several QA tasks, and the ablation with tool use disabled shows the gain comes mainly from tool calls, not fine-tuning alone.

Q11. (Pick all that are correct) 예상문제

Which of the following are correct about **ToolLLM's data construction pipeline** (Qin et al., ICLR 2024)?

- a. It collects 3,451 tools with 16,464 APIs by filtering RapidAPI, where one tool may contain multiple APIs.
- b. For instruction generation, all collected APIs are provided to ChatGPT simultaneously so that it can freely choose any combination.
- c. During solution path annotation, the sampled APIs are fed to ChatGPT as available functions, and two extra functions — "Finish with final answer" and "Finish by giving up" — are defined to terminate an action sequence.

- d. Depth-first search is used to explore action sequences efficiently, and only paths reaching a final answer are kept as golden data.

▼ 정답 & 해설 보기

정답: (a), (c), (d)

- (a) Correct. RapidAPI is an API marketplace with a tool $\supset$ API hierarchy; the numbers 3,451 / 16,464 are after filtering.
- (b) Incorrect. Showing 16K+ APIs at once is infeasible (each needs name, description, documentation in context); ToolLLM samples N APIs (S_N^{sub}) at a time, and instructions involve subsets $S_*^{sub} \subset S_N^{sub}$.
- (c) Correct. ChatGPT's function-call feature is used with APIs given at system-message level, plus the two termination functions.
- (d) Correct. DFSDT enables backtracking over branches; "give up" sequences are removed, leaving only validated (instruction, API, solution path) golden data.

Q12. (Pick all that are correct) 예상문제

Which of the following are correct about ToolLLM's retriever and evaluation?

- a. The API retriever is a BERT-based dense retriever (similar to DPR) trained with instructions as queries and API documents as passages.
- b. The trained retriever underperforms OpenAI's Ada embedding model because Ada was trained on far more data.
- c. The Pass metric measures whether the LLM can successfully complete the instruction, while the Win metric compares solution quality against ChatGPT.
- d. ToolLLaMA is obtained by fine-tuning LLaMA on the instruction-solution path pairs and achieves performance comparable to ChatGPT in some scenarios.

▼ 정답 & 해설 보기

정답: (a), (c), (d)

- (a) Correct. Query = instruction, passage = API document; negatives are APIs used by other instructions.

- (b) Incorrect. The fine-tuned retriever **outperforms** both BM25 and Ada — tool calling has special properties for which domain-specific fine-tuning helps a lot.
- (c) Correct. Pass = answerability (completing the instruction); Win = preference comparison against ChatGPT's solution.
- (d) Correct. ToolLLaMA (fine-tuned LLaMA-7B) matches ChatGPT-level performance in some scenarios.

Q13. (Pick all that are correct) 예상문제

Which of the following correctly contrast **Toolformer** and **ToolLLM**?

- Toolformer is self-supervised (filtering by loss reduction), whereas ToolLLM relies on a strong closed LLM (ChatGPT) for instruction generation and solution path annotation.
- Toolformer uses 5 tools while ToolLLM covers 16,000+ real-world APIs, because Toolformer's augmentation cost grows with the number of tools.
- Both methods include an API retrieval step at inference time to select relevant tools for a given query.
- ToolLLM's inference pipeline is analogous to RAG: retrieving relevant APIs corresponds to document retrieval, and generating actions with the APIs corresponds to generation.

▼ 정답 & 해설 보기

정답: (a), (b), (d)

- (a) Correct. This is the core methodological difference.
- (b) Correct. The per-tool augmentation cost is exactly why Toolformer used only 5 tools and why ToolLLM was motivated to scale to real-world APIs.
- (c) Incorrect. Only **ToolLLM** has a retrieval step (top-K APIs via the trained dense retriever). Toolformer trains all 5 tools into the model and needs no retrieval.

- (d) Correct. The lecture explicitly drew this RAG analogy for ToolLLM's two-stage inference.

3분 요약

개념	한 줄 정리
Tool use 동기	계산·실시간 정보·외부 행동은 LLM 단독으로 불가(hallucination 위험); tool use는 agent의 핵심 기능(+ memory, planning & action)이고 retrieval(RAG)도 tool의 일종
Prompting 기반 (function calling)	tools 인자 → system message의 자연어 설명 → implicit in-context learning으로 호출 결정. 모델은 이름+인자만 반환, 실행은 개발자 측. 유연하지만 large LLM에서만 신뢰 가능. Claude skill.md와 동일 구성(차이는 API 유무)
Toolformer 핵심	Self-supervised: "useful tool → helpful to predict next token". $c = (a_c, i_c)$, 특수 토큰 $\langle \text{API} \rangle, \langle / \text{API} \rangle, \rightarrow$ 로 텍스트화
Toolformer 5단계	① in-context로 호출 샘플링 ② 실행 ③ filtering: $L_i^- - L_i^+ \geq \tau, L_i^- = \min(L_i(\varepsilon), L_i(e(c_i, \varepsilon)))$ ④ 표준 LM loss로 fine-tuning ⑤ 추론 시 → 에서 멈추고 API 결과 삽입
Toolformer 결과/한계	GPT-J 6B + 5 tools가 GPT-3 능가(이득은 tool call에서); 그러나 비용이 도구 수에 비례 → real-world scale 불가
ToolLLM 데이터	RapidAPI에서 3,451 tools/16,464 APIs → ChatGPT로 instruction 생성(N개 subsample, few-shot 12/36, ~200k pairs) → solution path annotation(DFSĐT, Finish/give-up)으로 golden data만 유지
ToolLLM 추론/평가	RAG 유사 2단계(API retrieval → generation); BERT dense retriever가 BM25-Ada 능가; ToolLLaMA(LLaMA-7B)는 ChatGPT 필적; Pass=완수율, Win=ChatGPT 대비 품질; retrieved top-5 > ground-truth (diversity)
한 줄 대비	Toolformer = loss로 거르는 self-supervised 5-tool 증강 / ToolLLM = ChatGPT가 풀어서 거르는 real-world 16K-API instruction tuning + DFSĐT

[← 전체 목차](#)

[다음 장 →](#)